Neurolabs Android SDK Integration Guide

Technical Integration Specification

Document ID: NLB-ANDROID-INTEGRATION-GUIDE
Version: v1.1.7
Date: 2026-03-18
Status: Partner integration reference
Owner: Neurolabs

1. Scope

This guide is for partner Android apps integrating ‘neurolabs-android-sdk‘ directly.

2. Changes In v1.1.7

? No public API contract break.

? Default task routing config uses ‘taskUUID‘.

? Recommended custom-camera shelf payload remains strict shelf guidance with ‘validationPreset = IOS_PARITY‘.

? ‘liveQualityChecksEnabled‘ remains the key switch for pill/rotation/warning/error guidance behavior.

? ‘autoCloseAfterCapture = true‘ remains the recommended parity flow for single accepted captures.

3. Requirements

? ‘minSdk 26‘

? JDK 17

? Kotlin 2.x

4. Install

```
// app/build.gradle.kts
dependencies {
implementation(files("libs/neurolabs-android-sdk-v1.1.x.aar"))
}
```

If you consume a local ‘.aar‘, include required runtime deps (Compose/CameraX/serialization/etc.) or use the provided integration gradle script from the Cordova plugin setup.

5. SDK Initialization + Warmup

```
import ai.neurolabs.sdk.core.*
import kotlinx.coroutines.launch

class MainApp : Application() {
override fun onCreate() {
super.onCreate()

val config = NLConfiguration.production(
apiKey = BuildConfig.NL_API_KEY,
taskUUID = BuildConfig.NL_TASK_UUID,
debugLogging = BuildConfig.DEBUG
)

NeurolabsSDKCore.configure(this, config)
}
}

// e.g. in first Activity/VM
val sdk = NeurolabsSDKCore.shared ?: return
lifecycleScope.launch {
sdk.initialize(
modelFileName = "weights_litert.tflite",
labelsFileName = "labels.json"
)
sdk.waitForFullyLoaded()
}
```

6. Queue Management

```
val sdk = NeurolabsSDKCore.shared ?: return

// runtime queue behavior
sdk.setAutoSyncEnabled(true)
sdk.setWifiOnlyUploadsEnabled(false)
sdk.setDeletePhotosOnUploadSuccess(true)
sdk.setMaxQueueSize(200)
sdk.setUploadRetryCount(5)

// lifecycle actions
lifecycleScope.launch {
sdk.flushQueue()
sdk.retryFailedUploads()
}

val status = sdk.getQueueStatus()
println("pending=${status.pendingCount} failed=${status.failedCount}")
```

7. Open Custom Camera (Native Capture UI)

```
import ai.neurolabs.sdk.models.*
import ai.neurolabs.sdk.ui.openNativeCaptureUI

val sessionId = UUID.randomUUID().toString()

val config = NLNativeCaptureConfig(
guidanceMode = NLCaptureGuidanceMode.STRICT,
validationPreset = NLValidationPreset.IOS_PARITY,
confidenceThreshold = 0.25f,
iouThreshold = 0.45f,
maxCaptures = 1,
showAlignmentGuidance = true,
enableValidation = true,
autoCloseAfterCapture = true,
showDetections = false,
showCapturedRegions = false,
liveQualityChecksEnabled = true,
liveQualityTargetFps = 6
)

val options = NLNativeCaptureOptions(
type = NLShelfType.SHELF,
taskUUID = "<TASK_UUID_FOR_THIS_SESSION>",
sendDetectionsMetadata = true,
enablePreviewCropping = true,
maxImageDimension = 1920,
imageCompressionQuality = 0.85f
)

sdk.openNativeCaptureUI(
context = this,
sessionId = sessionId,
config = config,
options = options
)
```

Recommended capture payload notes:

? ‘liveQualityChecksEnabled = true‘ is the key for pill/rotation/warning/error guidance behavior.

? Keep ‘type = NLShelfType.SHELF‘ and ‘guidanceMode = NLCaptureGuidanceMode.STRICT‘ for full shelf guidance checks.

? Keep ‘showDetections = false‘ and ‘showCapturedRegions = false‘ to stay on the custom-camera guidance UI path.

? ‘guidanceMode = NLCaptureGuidanceMode.GUIDANCE‘ should only be used as a temporary fallback if a partner specifically needs looser Android behavior while a strict-guidance regression is being fixed.

8. Post-Processing Hooks

Use Activity Result API and process captures when camera returns.

```
private val captureLauncher = registerForActivityResult(
NLCaptureActivity.Contract()
) { result ->
when (result) {
is NLCaptureActivityResult.Success -> {
// capture metadata is in result.payload
val count = result.payload.captures.size
Log.d("NLB", "camera returned with $count captures")
}
is NLCaptureActivityResult.Cancelled -> {
Log.d("NLB", "camera cancelled")
}
is NLCaptureActivityResult.Error -> {
Log.e("NLB", "camera error: ${result.error.message}")
}
}
}
```

9. Delegate/Event Callbacks

```
sdk.delegate = object : NeurolabsSDKDelegate {
override fun onCaptureResult(result: NLNativeCaptureResult, captureId: String, sessionId:
String) {}
override fun onError(error: NLError, sessionId: String?, messageId: String?) {}
override fun onQueueStatusChange(status: NLQueueStatus) {}
override fun onLoadingProgress(progress: NLLoadingProgress) {}
override fun onQueueItemEnqueued(item: NLQueuedItem) {}
override fun onUploadSucceeded(item: NLQueuedItem) {}
override fun onUploadFailed(item: NLQueuedItem, error: NLError) {}
}
```

10. Notes

? Per-session upload routing is supported with ‘options.taskUUID‘.

? ‘autoCloseAfterCapture=true‘ closes after Save from preview/review flow.

? For open-ended shelf scanning today, prefer ‘maxCaptures = 1‘ with auto-close and reopen per accepted image; Android currently derives the shelf "Done" threshold from ‘maxCaptures‘.

? For partner apps, avoid passing base64 images across process boundaries.